

Product Requirement: B2C E-Commerce Platform MVP

เอกสารนี้ออกแบบมาเพื่อใช้เป็นโจทย์ทดสอบ workflow แบบ agentic AI ระหว่าง `claude-code`, `codex-cli` หรือ coding agents อื่น ๆ โดยเป้าหมายสุดท้ายคือให้ agent สร้าง output ได้ครบชุด ได้แก่ source-code repository, user stories, system architecture document, API specification, database design, test plan และเอกสารเสริมสำหรับ 1 MVP ที่ใช้งานได้จริง

1. Product Overview

1.1 Product Name

ShopPilot MVP

ข้อเสนอสำหรับ e-commerce platform แบบ B2C single-merchant โดยออกแบบให้รองรับการขายสินค้าออนไลน์ ตั้งแต่การค้นหาสินค้า เลือกสินค้า ใส่ตะกร้า สั่งซื้อ ชำระเงินแบบ mock ติดตามสถานะคำสั่งซื้อ และให้ admin จัดการ catalog, stock, order และ promotion ได้

1.2 Product Vision

สร้าง e-commerce MVP ที่มี scope พอสำหรับการทดสอบ end-to-end software delivery workflow โดยต้องมี business rules มากพอให้ agent ต้องคิดเรื่อง domain, state transition, data consistency, API contract, validation, security, testing และ documentation ไม่ใช่แค่ CRUD application

1.3 Business Goal

1. ลูกค้าสามารถเลือกซื้อสินค้าออนไลน์ได้ตั้งแต่ browse → cart → checkout → payment → order tracking
2. Admin สามารถจัดการสินค้า, stock, promotion และ order ได้จาก back office
3. ระบบต้องมี architecture ที่ยืดหยุ่นได้ในอนาคต เช่น แยก service, เพิ่ม payment gateway จริง, เพิ่ม shipping provider, เพิ่ม loyalty program
4. ใช้เป็น benchmark สำหรับทดสอบความสามารถของ AI coding agents ในการสร้าง software repo แบบ production-like MVP

1.4 MVP Positioning

MVP นี้ไม่ใช่ marketplace หลายร้านค้า แต่เป็น **single merchant e-commerce platform** ที่มีโครงสร้าง domain ชัดเจนและพร้อมขยายไปเป็น multi-vendor หรือ microservices ในอนาคต

2. Agentic AI Workflow Objective

2.1 Primary Objective

ใช้ requirement นี้เป็น input ให้ AI agents สร้าง software delivery artifacts ต่อไปนี้

Artifact	Required	Description
Source-code repo	Yes	Backend, frontend หรือ full-stack repo ที่ run ได้จริง
<code>README.md</code>	Yes	วิธี setup, run, test, seed data, env config
User stories	Yes	Markdown แยกตาม epic พร้อม acceptance criteria
System architecture document	Yes	Context, container, component, sequence, trade-off
API specification	Yes	OpenAPI 3.0 หรือ Markdown API spec
Database schema	Yes	ERD, migration, seed data
Test plan	Yes	Unit, integration, E2E, API contract test
ADR	Optional but recommended	Architecture Decision Records
Runbook	Optional but recommended	How to operate, troubleshoot, monitor
Security checklist	Optional but recommended	Auth, input validation, rate limit, sensitive log

2.2 Suggested AI Agent Roles

Agent Role	Suggested Tool	Responsibility
Product Analyst	Claude / GPT	แตก requirement เป็น user stories, flows, edge cases
Solution Architect	Claude / GPT	ออกแบบ architecture, data model, API boundary
Backend Engineer	Codex CLI / Claude Code	สร้าง backend service, API, DB migration, tests
Frontend Engineer	Codex CLI / Claude Code	สร้าง web UI, state management, integration
QA Engineer	Claude / Codex	สร้าง test plan, test cases, API tests
Reviewer	GPT / Claude	review repo, architecture, security, test coverage

2.3 Benchmark Criteria

ใช้ requirement นี้วัด agent ได้จาก

1. สร้าง source-code ที่ run ได้จริงด้วย Docker Compose
2. API behavior ตรงกับ requirement และ state transition
3. มี schema/migration/seed data ครบ
4. มี error handling และ validation ที่ชัดเจน
5. มี test coverage สำหรับ critical business flow
6. มีเอกสาร architecture และ API ที่สอดคล้องกับ code จริง
7. มี commit/task breakdown ที่ตรวจสอบความคืบหน้าได้
8. ไม่ over-engineer เกิน MVP แต่ยังต่อยอดได้

3. Assumptions

3.1 Product Assumptions

1. Platform เป็น B2C single merchant
2. ลูกค้าทั่วไปสามารถ browse สินค้าได้โดยไม่ต้อง login
3. การ checkout ต้อง login
4. Payment เป็น mock payment provider ใน MVP
5. Shipping เป็น mock shipping provider ใน MVP
6. Admin เป็น internal user ที่ login ผ่าน role-based access
7. สกุลเงินหลักคือ THB
8. ระบบรองรับภาษาเดียวก่อนคือ English หรือ Thai แต่ data model ต้องไม่ปิดทาง i18n
9. MVP รองรับ web application เป็น primary channel
10. Mobile app ยังไม่อยู่ใน MVP

3.2 Technical Assumptions

Agent สามารถเลือก stack ได้ แต่เพื่อ benchmark ที่ชัดเจน แนะนำ baseline นี้

Layer	Recommended Stack
Frontend	React / Next.js / Vite React
Backend	Go Gin, Node.js NestJS, หรือ Java Spring Boot
Database	PostgreSQL
Cache	Redis optional
Auth	JWT access token + refresh token
API Spec	OpenAPI 3.0
Local Runtime	Docker Compose

Layer	Recommended Stack
Testing	Unit + integration + API tests
CI	GitHub Actions optional

หากต้องการทดสอบความสามารถด้าน DDD/CQRS ให้ใช้ **modular monolith** ก่อน แล้วออกแบบ boundary ให้สามารถแยก service ภายหลังได้

4. MVP Scope

4.1 In Scope

MVP ต้องมี module ต่อไปนี้

1. Authentication & Authorization
2. Customer Profile
3. Product Catalog
4. Category Management
5. Product Search & Filter
6. Inventory Management
7. Shopping Cart
8. Checkout
9. Order Management
10. Mock Payment
11. Mock Shipping
12. Promotion / Coupon
13. Product Review
14. Admin Back Office
15. Notification Log
16. Audit Log
17. Basic Observability

4.2 Out of Scope

สิ่งต่อไปนี้ไม่อยู่ใน MVP แต่ควรออกแบบให้ต่อยอดได้

1. Real payment gateway integration
2. Real shipping provider integration
3. Multi-vendor marketplace
4. Loyalty point system
5. Installment / BNPL
6. Return / refund flow แบบเต็ม
7. Warehouse management system
8. Advanced recommendation engine
9. Real-time chat

10. Native mobile app
 11. Multi-currency
 12. Multi-language UI แบบสมบูรณ์
 13. AI product recommendation
 14. Accounting integration
-

5. User Roles

5.1 Guest User

ผู้ใช้ที่ยังไม่ login

สามารถทำได้

1. ดูหน้า home
2. ดูรายการสินค้า
3. ค้นหาและ filter สินค้า
4. ดูรายละเอียดสินค้า
5. สมัครสมาชิก
6. Login

ไม่สามารถทำได้

1. Checkout
2. เขียน review
3. ดู order history
4. ใช้ coupon เฉพาะสมาชิก

5.2 Customer

ผู้ใช้ที่สมัครสมาชิกและ login แล้ว

สามารถทำได้

1. จัดการ profile
2. เพิ่มสินค้าเข้าตะกร้า
3. ปรับจำนวนสินค้าในตะกร้า
4. Checkout
5. ใช้ coupon
6. ชำระเงินผ่าน mock payment
7. ดู order history
8. ดู order detail
9. ยกเลิก order ตามเงื่อนไข
10. เขียน review หลัง order ถูก delivered

5.3 Admin

ผู้ดูแลระบบร้านค้า

สามารถทำได้

1. จัดการสินค้า
2. จัดการ category
3. จัดการ stock
4. จัดการ promotion/coupon
5. ดูรายการ order ทั้งหมด
6. เปลี่ยนสถานะ order
7. ดู customer list
8. ดู audit log
9. ดู notification log

5.4 System

ระบบอัตโนมัติหรือ background job

สามารถทำได้

1. หมดอายุ cart item หรือ stock reservation
2. update payment status mock
3. update shipping status mock
4. สร้าง notification log
5. สร้าง audit log

6. Domain Boundaries

6.1 Recommended Bounded Contexts

Bounded Context	Responsibility	Core / Supporting
Identity	Authentication, user, role, token	Supporting
Catalog	Product, category, product visibility, review summary	Core
Inventory	Stock quantity, reservation, release, adjustment	Core
Cart	Customer cart, cart item, pricing snapshot	Supporting
Checkout	Validate cart, address, coupon, create order	Core
Order	Order state, order item, cancellation, fulfillment status	Core
Payment	Mock payment intent, payment status	Supporting

Bounded Context	Responsibility	Core / Supporting
Shipping	Mock shipment, tracking status	Supporting
Promotion	Coupon validation, discount calculation	Supporting
Notification	Notification log, email simulation	Supporting
Audit	Admin action log, critical state change log	Supporting

6.2 Modular Monolith Package Suggestion

```
shop-pilot/
  apps/
    backend/
    frontend/
  docs/
    architecture/
    api/
    adr/
    user-stories/
    test-plan/
  infra/
    docker-compose.yml
    postgres/
  scripts/
  README.md
```

Backend structure example

```
backend/
  cmd/server/
  internal/
    identity/
    catalog/
    inventory/
    cart/
    checkout/
    order/
    payment/
    shipping/
    promotion/
    notification/
    audit/
    shared/
```

```
migrations/  
tests/
```

แต่ละ module ควรแยกชั้นโดยประมาณ

```
module/  
  domain/  
  application/  
  infrastructure/  
  interfaces/http/
```

7. High-Level User Journeys

7.1 Browse to Purchase Journey

1. Guest เปิดหน้า home
2. Guest ดู product listing
3. Guest filter สินค้าตาม category, price range, availability
4. Guest เปิด product detail
5. Guest login หรือสมัครสมาชิก
6. Customer เพิ่มสินค้าเข้าตะกร้า
7. Customer ปรับจำนวนสินค้า
8. Customer กรอก shipping address
9. Customer ใส่ coupon ถ้ามี
10. System validate stock และ reserve stock
11. System สร้าง order draft หรือ pending order
12. Customer ยืนยัน checkout
13. System สร้าง payment intent แบบ mock
14. Customer เลือก payment result: success / failed / timeout
15. ถ้า payment success → order status เป็น paid
16. System สร้าง mock shipment
17. Admin หรือ system update shipment status
18. Customer ดู order tracking
19. หลัง delivered customer เขียน review ได้

7.2 Admin Catalog Management Journey

1. Admin login
2. Admin สร้าง category
3. Admin สร้างสินค้าใหม่
4. Admin upload หรือระบุ image URL
5. Admin set price, SKU, stock, visibility
6. System validate SKU ห้ามซ้ำ
7. Product ถูก publish

8. Customer เห็นสินค้าใน listing
9. Admin แก้ไข price หรือปิด visibility ได้
10. System เก็บ audit log ทุกครั้งที่ admin เปลี่ยนข้อมูลสำคัญ

7.3 Order Fulfillment Journey

1. Customer payment success
2. Order status เป็น PAID
3. Admin เห็น order ใหม่ใน dashboard
4. Admin เปลี่ยน order เป็น PACKING
5. Admin เปลี่ยน order เป็น SHIPPED พร้อม tracking number
6. System สร้าง shipment record
7. Customer เห็น tracking status
8. Admin หรือ mock system เปลี่ยนเป็น DELIVERED
9. Customer เขียน review ได้

8. Functional Requirements

8.1 Authentication & Authorization

ID	Requirement	Priority	Acceptance Criteria
AUTH-001	Guest สามารถสมัครสมาชิกด้วย email, password, name	Must	Email ต้อง unique, password ถูก hash, default role เป็น CUSTOMER
AUTH-002	User สามารถ login ด้วย email/ password	Must	ถ้าถูกต้อง return access token และ refresh token
AUTH-003	ระบบต้องแยก role CUSTOMER และ ADMIN	Must	Admin endpoint ต้อง reject customer token
AUTH-004	User สามารถ logout ได้	Should	Refresh token ถูก revoke หรือ invalidated
AUTH-005	Token ต้องมี expiry	Must	Access token อายุสั้น, refresh token อายุยาวกว่า
AUTH-006	Password ต้องไม่ถูกเก็บเป็น plain text	Must	ใช้ bcrypt หรือ algorithm ที่เหมาะสม
AUTH-007	Login failure ต้องไม่บอกว่า email หรือ password ผิดส่วนใด	Must	Return generic error

8.2 Customer Profile

ID	Requirement	Priority	Acceptance Criteria
CUST-001	Customer ดู profile ตัวเองได้	Must	แสดง name, email, phone, default address
CUST-002	Customer แก้ไข name, phone ได้	Must	Email แก้ไม่ได้ใน MVP
CUST-003	Customer เพิ่ม shipping address ได้	Must	ต้องมี receiver name, phone, address line, province, district, postal code
CUST-004	Customer เลือก default address ได้	Should	Checkout ใช้ default address เป็นค่าเริ่มต้น
CUST-005	Customer ลบ address ได้ถ้าไม่ถูกใช้ใน active order	Could	ถ้า order ยัง active ห้ามลบ

8.3 Product Catalog

ID	Requirement	Priority	Acceptance Criteria
CAT-001	Guest/Customer ดูรายการสินค้าได้	Must	แสดงเฉพาะสินค้า status ACTIVE และ <code>visible = true</code>
CAT-002	Product listing ต้องมี pagination	Must	รองรับ page, limit, sort
CAT-003	Product listing ต้อง filter ตาม category ได้	Must	ส่ง categoryId แล้วได้สินค้าของ category นั้น
CAT-004	Product listing ต้อง filter ตาม price range ได้	Should	minPrice/maxPrice
CAT-005	Product listing ต้อง filter ตาม availability ได้	Should	inStock=true แสดงเฉพาะสินค้ามี stock
CAT-006	Product detail แสดงข้อมูลครบ	Must	name, description, images, price, stock status, category, review summary
CAT-007	Admin สร้างสินค้าได้	Must	SKU unique, price > 0, category valid
CAT-008	Admin แก้ไขสินค้าได้	Must	เก็บ audit log เมื่อแก้ price/status/stock policy
CAT-009	Admin soft delete สินค้าได้	Must	Product ไม่หายจาก order history เดิม
CAT-010	Product ต้องมี status	Must	DRAFT, ACTIVE, INACTIVE, DELETED

8.4 Category Management

ID	Requirement	Priority	Acceptance Criteria
CATE-001	Admin สร้าง category ได้	Must	name unique ใน level เดียวกัน
CATE-002	Admin แก้ไข category ได้	Must	slug ต้อง unique
CATE-003	Admin ปิด category ได้	Should	สินค้าใน category นั้นยังคงอยู่แต่ filter ไม่แสดง category inactive
CATE-004	Category รองรับ parent-child 1 level	Could	เช่น Electronics > Headphones

8.5 Inventory

ID	Requirement	Priority	Acceptance Criteria
INV-001	ระบบต้องเก็บ stock quantity ต่อ SKU	Must	มี available, reserved, sold
INV-002	เมื่อเพิ่มสินค้าเข้าตะกร้า ยังไม่ reserve stock	Must	reserve เฉพาะตอน checkout
INV-003	ตอน checkout ต้อง reserve stock	Must	ถ้า stock ไม่พอ checkout ต้อง fail
INV-004	Stock reservation มี expiry	Should	ถ้า payment ไม่สำเร็จภายในเวลาที่กำหนด stock ถูก release
INV-005	Payment success ต้อง convert reserved → sold	Must	available ลด, reserved ลด, sold เพิ่ม
INV-006	Payment failed ต้อง release reserved stock	Must	available กลับมาเท่าเดิม
INV-007	Admin ปรับ stock ได้	Must	ต้องสร้าง stock adjustment record
INV-008	ห้าม stock ติดลบ	Must	ทุก operation ต้อง validate

8.6 Shopping Cart

ID	Requirement	Priority	Acceptance Criteria
CART-001	Customer เพิ่มสินค้าเข้าตะกร้าได้	Must	ต้อง login, product active, quantity > 0
CART-002	Customer แก่จำนวนสินค้าในตะกร้า ได้	Must	quantity ต้องไม่เกิน available stock ณ เวลานั้น
CART-003	Customer ลบสินค้าออกจากตะกร้าได้	Must	Cart total update ถูกต้อง
CART-004	Cart ต้องคำนวณ subtotal ได้	Must	ใช้ราคาปัจจุบันหรือ pricing snapshot ตาม design ที่เลือก

ID	Requirement	Priority	Acceptance Criteria
CART-005	ถ้า product inactive ต้องแจ้งใน cart	Should	Item ไม่สามารถ checkout ได้
CART-006	Cart item duplicate SKU ต้อง merge quantity	Must	เพิ่ม item เดิมซ้ำแล้วรวมจำนวน
CART-007	Cart ต้องมี updatedAt	Must	ใช้ตรวจสอบ stale cart

8.7 Promotion / Coupon

ID	Requirement	Priority	Acceptance Criteria
PROMO-001	Admin สร้าง coupon ได้	Must	code unique, type, value, start/end date
PROMO-002	Coupon รองรับ fixed amount discount	Must	เช่น ลด 100 บาท
PROMO-003	Coupon รองรับ percentage discount	Should	เช่น ลด 10%, ต้องมี max discount ได้
PROMO-004	Coupon มี minimum order amount	Must	ถ้า subtotal ไม่ถึงใช้ไม่ได้
PROMO-005	Coupon มี usage limit รวม	Should	เช่น ใช้ได้ 100 ครั้ง
PROMO-006	Coupon มี per-customer limit	Should	เช่น ลูกค้าคนหนึ่งใช้ได้ 1 ครั้ง
PROMO-007	Checkout ต้อง validate coupon อีกครั้ง	Must	ไม่ใช่ผล validate จาก frontend เป็น source of truth
PROMO-008	Discount ต้องไม่ทำให้ total ต่ำกว่า 0	Must	total min = 0

8.8 Checkout

ID	Requirement	Priority	Acceptance Criteria
CHK-001	Customer checkout จาก cart ได้	Must	ต้องมี item อย่างน้อย 1 ชิ้น
CHK-002	Checkout ต้อง validate address	Must	address ต้องครบและเป็นของ customer คนนั้น
CHK-003	Checkout ต้อง validate product status	Must	inactive/deleted product checkout ไม่ได้
CHK-004	Checkout ต้อง validate stock	Must	ถ้า stock ไม่พอ ต้องคืน error ราย item
CHK-005	Checkout ต้อง calculate price server-side	Must	ห้าม trust total จาก client

ID	Requirement	Priority	Acceptance Criteria
CHK-006	Checkout ต้อง apply coupon server-side	Must	discount คำนวณใหม่เสมอ
CHK-007	Checkout success ต้องสร้าง order	Must	Order status เริ่มต้นเป็น <code>PENDING_PAYMENT</code>
CHK-008	Checkout ต้อง reserve stock	Must	reservation ผูกกับ order
CHK-009	Checkout ต้อง idempotent ได้บางระดับ	Should	client ส่ง idempotency key เพื่อกันกดซ้ำ
CHK-010	หลัง checkout แล้ว cart ต้องถูก clear เฉพาะ item ที่สั่งซื้อสำเร็จ	Must	cart empty หรือ remaining invalid item ตาม design

8.9 Order Management

ID	Requirement	Priority	Acceptance Criteria
ORD-001	Customer ดู order history ของตัวเองได้	Must	ไม่เห็น order ของคนอื่น
ORD-002	Customer ดู order detail ได้	Must	แสดง item, price snapshot, address snapshot, status timeline
ORD-003	Order ต้องมี state machine	Must	State transition ต้อง validate
ORD-004	Customer cancel order ได้ก่อน payment success	Must	จาก <code>PENDING_PAYMENT</code> → <code>CANCELLED</code>
ORD-005	Admin cancel order ได้ก่อน shipped	Should	ต้องใส่ reason
ORD-006	Admin update order status ได้	Must	<code>PAID</code> → <code>PACKING</code> → <code>SHIPPED</code> → <code>DELIVERED</code>
ORD-007	Order item ต้องเก็บ price snapshot	Must	เปลี่ยนราคาสินค้าที่หลังไม่กระทบ order เก่า
ORD-008	Order ต้องมี running order number	Should	เช่น <code>ORD-20260507-000001</code>
ORD-009	Order ต้องมี status history	Must	บันทึกทุก transition พร้อม timestamp

8.10 Payment Mock

ID	Requirement	Priority	Acceptance Criteria
PAY-001	Checkout ต้องสร้าง payment intent	Must	payment status เริ่มเป็น <code>REQUIRES_PAYMENT</code>

ID	Requirement	Priority	Acceptance Criteria
PAY-002	Customer simulate payment success ได้	Must	order status เปลี่ยนเป็น PAID
PAY-003	Customer simulate payment failed ได้	Must	order status เป็น PAYMENT_FAILED , stock released
PAY-004	Customer simulate payment timeout ได้	Should	order เป็น PAYMENT_EXPIRED , stock released
PAY-005	Payment callback ต้อง idempotent	Must	callback ซ้ำไม่ทำให้ stock หรือ order เพี้ยน
PAY-006	Payment amount ต้องตรงกับ order total	Must	ถ้า amount mismatch ต้อง reject
PAY-007	Payment transaction ต้องเก็บ reference	Must	mockPaymentRef, providerStatus, paidAt

8.11 Shipping Mock

ID	Requirement	Priority	Acceptance Criteria
SHIP-001	เมื่อ order paid ระบบสร้าง shipment draft	Must	shipment status READY_TO_PACK
SHIP-002	Admin mark order packing ได้	Must	shipment status PACKING
SHIP-003	Admin mark order shipped ได้	Must	ต้องมี tracking number
SHIP-004	Customer ดู tracking number ได้	Must	เฉพาะ order ของตัวเอง
SHIP-005	Admin mark delivered ได้	Must	order status DELIVERED
SHIP-006	Shipment ต้องมี status history	Should	บันทึก timeline

8.12 Product Review

ID	Requirement	Priority	Acceptance Criteria
REV-001	Customer เขียน review ได้เมื่อ order delivered	Must	ต้องซื้อสินค้านั้นจริง
REV-002	Review มี rating 1-5	Must	rating out of range ต้อง reject
REV-003	Customer เขียน review ต่อ order item ได้ 1 ครั้ง	Must	กัน duplicate review
REV-004	Review แสดงใน product detail	Should	แสดง average rating และ count
REV-005	Admin hide review ได้	Could	Soft hide พร้อม reason

8.13 Admin Back Office

ID	Requirement	Priority	Acceptance Criteria
ADM-001	Admin login เข้าหลังบ้านได้	Must	ใช้ role ADMIN
ADM-002	Admin dashboard เห็น summary ได้	Should	total order, revenue, pending shipment, low stock
ADM-003	Admin search order ได้	Must	ค้นด้วย order number, customer email, status
ADM-004	Admin filter order ตาม status/ date ได้	Must	รองรับ pagination
ADM-005	Admin ดู customer list ได้	Should	ไม่แสดง password/hash/token
ADM-006	Admin ดู audit log ได้	Should	แสดง actor, action, entity, before/after summary

8.14 Notification Log

ID	Requirement	Priority	Acceptance Criteria
NOTI-001	ระบบสร้าง notification log เมื่อ order created	Should	type <code>ORDER_CREATED</code>
NOTI-002	ระบบสร้าง notification log เมื่อ payment success	Should	type <code>PAYMENT_SUCCESS</code>
NOTI-003	ระบบสร้าง notification log เมื่อ shipped	Should	type <code>ORDER_SHIPPED</code>
NOTI-004	ไม่ต้องส่ง email จริงใน MVP	Must	เก็บ log แทน

8.15 Audit Log

ID	Requirement	Priority	Acceptance Criteria
AUD-001	Admin action สำคัญต้องมี audit log	Must	product update, stock adjustment, order status change
AUD-002	Audit log ต้องมี actor	Must	actorUserId, role
AUD-003	Audit log ต้องมี entity reference	Must	entityType, entityId
AUD-004	Audit log ต้องมี action	Must	CREATE_PRODUCT, UPDATE_STOCK, CHANGE_ORDER_STATUS
AUD-005	Audit log ต้องไม่เก็บ sensitive data	Must	password/token/card data ห้าม log

9. Order State Machine

9.1 Order Status

Status	Meaning
PENDING_PAYMENT	Order created and waiting for payment
PAID	Payment success, stock confirmed sold
PAYMENT_FAILED	Payment failed and stock released
PAYMENT_EXPIRED	Payment timeout and stock released
PACKING	Admin is preparing package
SHIPPED	Package shipped with tracking number
DELIVERED	Customer received package
CANCELLED	Order cancelled before completion

9.2 Allowed Transitions

From	To	Actor	Condition
PENDING_PAYMENT	PAID	Payment System	Payment success and amount matched
PENDING_PAYMENT	PAYMENT_FAILED	Payment System	Payment failed
PENDING_PAYMENT	PAYMENT_EXPIRED	System	Payment timeout
PENDING_PAYMENT	CANCELLED	Customer/ Admin	Before payment success
PAID	PACKING	Admin	Shipment exists
PACKING	SHIPPED	Admin	Tracking number required
SHIPPED	DELIVERED	Admin/System	Delivery confirmed
PAID	CANCELLED	Admin	Only before packing, reason required

9.3 Forbidden Transitions

1. DELIVERED → CANCELLED
2. SHIPPED → PAID
3. PAYMENT_FAILED → PAID using same payment transaction
4. PAYMENT_EXPIRED → PAID after stock released

5. `CANCELLED` → any active status

10. Inventory Rules

10.1 Stock Fields

Field	Meaning
<code>availableQty</code>	จำนวนที่ขายได้ ณ ตอนนี้
<code>reservedQty</code>	จำนวนที่ถูก lock ระหว่างรอ payment
<code>soldQty</code>	จำนวนที่ขายสำเร็จแล้ว

10.2 Stock Invariant

```
availableQty >= 0
reservedQty >= 0
soldQty >= 0
```

ถ้ามี total stock tracking

```
totalQty = availableQty + reservedQty + soldQty
```

10.3 Checkout Reservation Example

ก่อน checkout

```
Product A
availableQty = 10
reservedQty = 0
soldQty = 0
```

Customer checkout quantity 2

```
availableQty = 8
reservedQty = 2
soldQty = 0
```

Payment success

```
availableQty = 8
reservedQty = 0
soldQty = 2
```

Payment failed

```
availableQty = 10
reservedQty = 0
soldQty = 0
```

11. Pricing Rules

11.1 Price Calculation

Order total ต้องคำนวณฝั่ง server เท่านั้น

```
itemSubtotal = itemUnitPrice * quantity
subtotal = sum(itemSubtotal)
discount = couponDiscount(subtotal)
shippingFee = calculateShippingFee(subtotal, address)
grandTotal = subtotal - discount + shippingFee
```

11.2 MVP Shipping Fee Rule

เพื่อให้ MVP ง่ายแต่ยังมี business rule

Condition	Shipping Fee
subtotal >= 1500 THB	0 THB
subtotal < 1500 THB	60 THB

11.3 Coupon Rule

Coupon discount ต้องถูก apply หลัง subtotal และก่อน shipping fee

```
grandTotal = subtotal - discount + shippingFee
```

Discount ต้องไม่ทำให้ grandTotal ติดลบ

12. Data Model

12.1 Core Entities

User

Field	Type	Required	Notes
id	UUID	Yes	Primary key
email	string	Yes	Unique
passwordHash	string	Yes	Never expose
name	string	Yes	Display name
phone	string	No	Optional
role	enum	Yes	CUSTOMER, ADMIN
status	enum	Yes	ACTIVE, SUSPENDED
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

Address

Field	Type	Required	Notes
id	UUID	Yes	Primary key
userId	UUID	Yes	Owner
receiverName	string	Yes	
phone	string	Yes	
addressLine1	string	Yes	
addressLine2	string	No	
province	string	Yes	
district	string	Yes	
subDistrict	string	No	
postalCode	string	Yes	
isDefault	boolean	Yes	
createdAt	datetime	Yes	

Field	Type	Required	Notes
updatedAt	datetime	Yes	

Category

Field	Type	Required	Notes
id	UUID	Yes	Primary key
parentId	UUID	No	For nested category
name	string	Yes	
slug	string	Yes	Unique
status	enum	Yes	ACTIVE, INACTIVE
sortOrder	int	No	
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

Product

Field	Type	Required	Notes
id	UUID	Yes	Primary key
sku	string	Yes	Unique
categoryId	UUID	Yes	
name	string	Yes	
slug	string	Yes	Unique
description	text	Yes	
price	decimal	Yes	> 0
compareAtPrice	decimal	No	For showing discount
status	enum	Yes	DRAFT, ACTIVE, INACTIVE, DELETED
visible	boolean	Yes	
mainImageUrl	string	No	
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

ProductImage

Field	Type	Required	Notes
id	UUID	Yes	Primary key
productId	UUID	Yes	
imageUrl	string	Yes	
altText	string	No	
sortOrder	int	Yes	

Inventory

Field	Type	Required	Notes
id	UUID	Yes	Primary key
productId	UUID	Yes	Unique
availableQty	int	Yes	>= 0
reservedQty	int	Yes	>= 0
soldQty	int	Yes	>= 0
lowStockThreshold	int	No	
updatedAt	datetime	Yes	

StockAdjustment

Field	Type	Required	Notes
id	UUID	Yes	Primary key
productId	UUID	Yes	
adjustmentType	enum	Yes	INCREASE, DECREASE, CORRECTION
quantity	int	Yes	
reason	string	Yes	
actorUserId	UUID	Yes	Admin
createdAt	datetime	Yes	

Cart

Field	Type	Required	Notes
id	UUID	Yes	Primary key
userId	UUID	Yes	Unique active cart per customer
status	enum	Yes	ACTIVE, CHECKED_OUT, ABANDONED
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

CartItem

Field	Type	Required	Notes
id	UUID	Yes	Primary key
cartId	UUID	Yes	
productId	UUID	Yes	
quantity	int	Yes	> 0
addedAt	datetime	Yes	
updatedAt	datetime	Yes	

Coupon

Field	Type	Required	Notes
id	UUID	Yes	Primary key
code	string	Yes	Unique uppercase
discountType	enum	Yes	FIXED, PERCENTAGE
discountValue	decimal	Yes	
maxDiscountAmount	decimal	No	For percentage
minOrderAmount	decimal	No	
usageLimit	int	No	Overall limit
usedCount	int	Yes	
perCustomerLimit	int	No	
startAt	datetime	Yes	
endAt	datetime	Yes	

Field	Type	Required	Notes
status	enum	Yes	ACTIVE, INACTIVE

Order

Field	Type	Required	Notes
id	UUID	Yes	Primary key
orderNumber	string	Yes	Unique
userId	UUID	Yes	
status	enum	Yes	Order status
subtotal	decimal	Yes	
discountAmount	decimal	Yes	
shippingFee	decimal	Yes	
grandTotal	decimal	Yes	
couponCode	string	No	Snapshot
shippingAddressSnapshot	json	Yes	Snapshot
createdAt	datetime	Yes	
updatedAt	datetime	Yes	
paidAt	datetime	No	
cancelledAt	datetime	No	

OrderItem

Field	Type	Required	Notes
id	UUID	Yes	Primary key
orderId	UUID	Yes	
productId	UUID	Yes	
skuSnapshot	string	Yes	
nameSnapshot	string	Yes	
unitPrice	decimal	Yes	Price snapshot
quantity	int	Yes	
lineTotal	decimal	Yes	

OrderStatusHistory

Field	Type	Required	Notes
id	UUID	Yes	Primary key
orderId	UUID	Yes	
fromStatus	enum	No	Null for initial
toStatus	enum	Yes	
actorType	enum	Yes	CUSTOMER, ADMIN, SYSTEM
actorUserId	UUID	No	
reason	string	No	
createdAt	datetime	Yes	

PaymentTransaction

Field	Type	Required	Notes
id	UUID	Yes	Primary key
orderId	UUID	Yes	
provider	string	Yes	MOCK
providerRef	string	Yes	Unique
amount	decimal	Yes	
status	enum	Yes	REQUIRES_PAYMENT, SUCCESS, FAILED, EXPIRED
paidAt	datetime	No	
failedReason	string	No	
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

Shipment

Field	Type	Required	Notes
id	UUID	Yes	Primary key
orderId	UUID	Yes	
status	enum	Yes	READY_TO_PACK, PACKING, SHIPPED, DELIVERED
trackingNumber	string	No	Required when shipped

Field	Type	Required	Notes
carrier	string	No	MOCK_EXPRESS
shippedAt	datetime	No	
deliveredAt	datetime	No	
createdAt	datetime	Yes	
updatedAt	datetime	Yes	

Review

Field	Type	Required	Notes
id	UUID	Yes	Primary key
productId	UUID	Yes	
orderId	UUID	Yes	Unique
userId	UUID	Yes	
rating	int	Yes	1-5
comment	text	No	
visible	boolean	Yes	
createdAt	datetime	Yes	

AuditLog

Field	Type	Required	Notes
id	UUID	Yes	Primary key
actorUserId	UUID	No	
actorRole	string	No	
action	string	Yes	
entityType	string	Yes	
entityId	string	Yes	
beforeSummary	json	No	
afterSummary	json	No	
ipAddress	string	No	
createdAt	datetime	Yes	

NotificationLog

Field	Type	Required	Notes
id	UUID	Yes	Primary key
userId	UUID	Yes	
type	string	Yes	
channel	string	Yes	EMAIL MOCK
subject	string	Yes	
body	text	Yes	
status	enum	Yes	CREATED, SENT MOCK, FAILED
createdAt	datetime	Yes	

13. API Specification Draft

Base URL

/api/v1

Authentication

Authorization: Bearer <access_token>

13.1 Auth APIs

Method	Path	Auth	Description
POST	/auth/register	Public	Register customer
POST	/auth/login	Public	Login
POST	/auth/refresh	Refresh token	Refresh access token
POST	/auth/logout	User	Logout
GET	/auth/me	User	Get current user

POST /auth/register

Request

```
{
  "email": "customer@example.com",
  "password": "P@ssw0rd123",
  "name": "Demo Customer",
  "phone": "0812345678"
}
```

Response

```
{
  "userId": "uuid",
  "email": "customer@example.com",
  "name": "Demo Customer",
  "role": "CUSTOMER"
}
```

POST /auth/login

Request

```
{
  "email": "customer@example.com",
  "password": "P@ssw0rd123"
}
```

Response

```
{
  "accessToken": "jwt",
  "refreshToken": "jwt-or-random-token",
  "expiresIn": 900
}
```

13.2 Product APIs

Method	Path	Auth	Description
GET	/products	Public	List active products
GET	/products/{productId}	Public	Get product detail
GET	/categories	Public	List active categories

Method	Path	Auth	Description
POST	/admin/products	Admin	Create product
PUT	/admin/products/{productId}	Admin	Update product
PATCH	/admin/products/{productId}/status	Admin	Change product status
POST	/admin/categories	Admin	Create category

GET /products

Query params

Param	Type	Required	Example
page	int	No	1
limit	int	No	20
q	string	No	headphone
categoryId	UUID	No	uuid
minPrice	decimal	No	100
maxPrice	decimal	No	3000
inStock	boolean	No	true
sort	string	No	price_asc, price_desc, newest

Response

```
{
  "items": [
    {
      "id": "uuid",
      "sku": "SKU-001",
      "name": "Wireless Headphone",
      "price": 1290,
      "mainImageUrl": "https://example.com/img.jpg",
      "category": {
        "id": "uuid",
        "name": "Electronics"
      },
      "inStock": true,
      "averageRating": 4.5,
      "reviewCount": 12
    }
  ]
}
```

```
],
"page": 1,
"limit": 20,
"total": 100
}
```

POST `/admin/products`

Request

```
{
  "sku": "SKU-001",
  "categoryId": "uuid",
  "name": "Wireless Headphone",
  "slug": "wireless-headphone",
  "description": "Bluetooth headphone with noise reduction",
  "price": 1290,
  "compareAtPrice": 1590,
  "status": "ACTIVE",
  "visible": true,
  "mainImageUrl": "https://example.com/img.jpg",
  "initialStock": 100
}
```

13.3 Cart APIs

Method	Path	Auth	Description
GET	<code>/cart</code>	Customer	Get active cart
POST	<code>/cart/items</code>	Customer	Add item to cart
PATCH	<code>/cart/items/{cartItemId}</code>	Customer	Update quantity
DELETE	<code>/cart/items/{cartItemId}</code>	Customer	Remove item
DELETE	<code>/cart</code>	Customer	Clear cart

POST `/cart/items`

Request

```
{
  "productId": "uuid",
```

```
    "quantity": 2
  }
}
```

Response

```
{
  "cartId": "uuid",
  "items": [
    {
      "cartItemId": "uuid",
      "productId": "uuid",
      "name": "Wireless Headphone",
      "unitPrice": 1290,
      "quantity": 2,
      "lineTotal": 2580,
      "available": true
    }
  ],
  "subtotal": 2580
}
```

13.4 Checkout APIs

Method	Path	Auth	Description
POST	/checkout/preview	Customer	Validate cart and preview total
POST	/checkout/place-order	Customer	Create order and payment intent

POST /checkout/preview

Request

```
{
  "addressId": "uuid",
  "couponCode": "WELCOME100"
}
```

Response

```
{
  "valid": true,
  "items": [
    {
```

```
    "productId": "uuid",
    "sku": "SKU-001",
    "name": "Wireless Headphone",
    "unitPrice": 1290,
    "quantity": 2,
    "lineTotal": 2580
  }
],
"subtotal": 2580,
"discountAmount": 100,
"shippingFee": 0,
"grandTotal": 2480,
"coupon": {
  "code": "WELCOME100",
  "valid": true,
  "message": "Coupon applied"
}
}
```

POST /checkout/place-order

Headers

Idempotency-Key: customer-generated-key

Request

```
{
  "addressId": "uuid",
  "couponCode": "WELCOME100"
}
```

Response

```
{
  "orderId": "uuid",
  "orderNumber": "ORD-20260507-000001",
  "status": "PENDING_PAYMENT",
  "grandTotal": 2480,
  "payment": {
    "paymentTransactionId": "uuid",
    "provider": "MOCK",
    "providerRef": "MOCKPAY-123456",
    "status": "REQUIRES_PAYMENT"
  }
}
```

```
}  
}
```

13.5 Payment APIs

Method	Path	Auth	Description
POST	<code>/payments/{paymentTransactionId}/simulate-success</code>	Customer	Simulate success
POST	<code>/payments/{paymentTransactionId}/simulate-failed</code>	Customer	Simulate failed
POST	<code>/payments/{paymentTransactionId}/simulate-timeout</code>	Customer	Simulate timeout

POST `/payments/{paymentTransactionId}/simulate-success`

Request

```
{  
  "amount": 2480  
}
```

Response

```
{  
  "orderId": "uuid",  
  "orderNumber": "ORD-20260507-000001",  
  "orderStatus": "PAID",  
  "paymentStatus": "SUCCESS",  
  "paidAt": "2026-05-07T10:00:00Z"  
}
```

13.6 Order APIs

Method	Path	Auth	Description
GET	<code>/orders</code>	Customer	Customer order history
GET	<code>/orders/{orderId}</code>	Customer	Customer order detail
POST	<code>/orders/{orderId}/cancel</code>	Customer	Cancel pending order
GET	<code>/admin/orders</code>	Admin	Admin order search

Method	Path	Auth	Description
GET	/admin/orders/{orderId}	Admin	Admin order detail
PATCH	/admin/orders/{orderId}/status	Admin	Change order status

PATCH /admin/orders/{orderId}/status

Request

```
{
  "targetStatus": "SHIPPED",
  "trackingNumber": "MOCK-TRACK-0001",
  "carrier": "MOCK_EXPRESS",
  "reason": "Package shipped"
}
```

Response

```
{
  "orderId": "uuid",
  "orderNumber": "ORD-20260507-000001",
  "fromStatus": "PACKING",
  "toStatus": "SHIPPED",
  "shipment": {
    "trackingNumber": "MOCK-TRACK-0001",
    "carrier": "MOCK_EXPRESS",
    "status": "SHIPPED"
  }
}
```

13.7 Review APIs

Method	Path	Auth	Description
POST	/products/{productId}/reviews	Customer	Create review
GET	/products/{productId}/reviews	Public	List visible reviews
PATCH	/admin/reviews/{reviewId}/hide	Admin	Hide review

13.8 Admin Inventory APIs

Method	Path	Auth	Description
GET	/admin/inventory	Admin	List inventory

Method	Path	Auth	Description
POST	<code>/admin/inventory/{productId}/adjust</code>	Admin	Adjust stock

POST `/admin/inventory/{productId}/adjust`

Request

```
{
  "adjustmentType": "INCREASE",
  "quantity": 50,
  "reason": "Initial warehouse stock"
}
```

14. Error Response Standard

ทุก API ควรใช้ error shape เดียวกัน

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Some products do not have enough stock",
    "details": [
      {
        "productId": "uuid",
        "requestedQty": 5,
        "availableQty": 2
      }
    ],
    "requestId": "req-abc-123"
  }
}
```

14.1 Common Error Codes

Code	HTTP Status	Meaning
<code>VALIDATION_ERROR</code>	400	Invalid request
<code>UNAUTHORIZED</code>	401	Missing or invalid token
<code>FORBIDDEN</code>	403	Role not allowed
<code>NOT_FOUND</code>	404	Resource not found

Code	HTTP Status	Meaning
CONFLICT	409	Duplicate or invalid state conflict
INSUFFICIENT_STOCK	409	Stock not enough
INVALID_COUPON	400	Coupon cannot be used
INVALID_ORDER_STATE	409	State transition not allowed
PAYMENT_AMOUNT_MISMATCH	409	Payment amount mismatch
INTERNAL_ERROR	500	Unexpected error

15. User Stories

15.1 Epic: Authentication

US-AUTH-001 Register Customer

As a guest user, I want to register with my email and password so that I can purchase products.

Acceptance Criteria

1. Given I provide valid email, password, name and phone, when I register, then the system creates a customer account
2. Given I use an existing email, when I register, then the system returns duplicate email error
3. Given my password is too weak, when I register, then the system returns validation error
4. Password must be stored as hash

US-AUTH-002 Login

As a registered customer, I want to login so that I can access my cart and orders.

Acceptance Criteria

1. Given valid credentials, when I login, then I receive access token and refresh token
2. Given invalid credentials, when I login, then I receive generic authentication error
3. The response must not expose password hash or internal auth details

15.2 Epic: Product Discovery

US-CAT-001 Browse Product Listing

As a guest user, I want to browse products so that I can decide what to buy.

Acceptance Criteria

1. Product listing shows only active and visible products
2. Listing supports pagination
3. Listing includes name, price, image, category, stock status and rating summary
4. Deleted or inactive products are not visible

US-CAT-002 Search and Filter Products

As a customer, I want to search and filter products so that I can find products faster.

Acceptance Criteria

1. Search supports product name keyword
2. Filter supports category
3. Filter supports price range
4. Filter supports in-stock only
5. Sort supports newest, price ascending and price descending

US-CAT-003 View Product Detail

As a customer, I want to view product detail so that I can understand product information before buying.

Acceptance Criteria

1. Detail page shows name, SKU, description, price, images and stock status
2. Detail page shows average rating and review count
3. If product is inactive, public API should return not found or unavailable depending on design

15.3 Epic: Cart

US-CART-001 Add Product to Cart

As a customer, I want to add a product to my cart so that I can buy it later.

Acceptance Criteria

1. Customer must login before adding item
2. Product must be active and visible
3. Quantity must be greater than zero
4. Adding same product again merges quantity
5. Quantity cannot exceed available stock

US-CART-002 Update Cart Quantity

As a customer, I want to update item quantity in my cart so that I can control how many items I buy.

Acceptance Criteria

1. Customer can increase or decrease quantity
2. Quantity zero should remove item or return validation error depending on design
3. Quantity cannot exceed available stock
4. Cart subtotal updates correctly

15.4 Epic: Checkout

US-CHK-001 Preview Checkout

As a customer, I want to preview checkout total so that I know the final amount before placing order.

Acceptance Criteria

1. System validates cart item availability
2. System validates address
3. System validates coupon
4. System calculates subtotal, discount, shipping fee and grand total server-side
5. Response returns item-level pricing snapshot

US-CHK-002 Place Order

As a customer, I want to place an order from my cart so that I can proceed to payment.

Acceptance Criteria

1. Given valid cart and address, when I place order, then order is created with `PENDING_PAYMENT`
2. Stock is reserved for order items
3. Payment transaction is created with `REQUIRES_PAYMENT`
4. Cart is cleared after successful order creation
5. Repeated request with same idempotency key must not create duplicate order

15.5 Epic: Payment

US-PAY-001 Simulate Payment Success

As a customer, I want to simulate successful payment so that I can complete my order in MVP.

Acceptance Criteria

1. Payment amount must match order grand total
2. Payment status becomes `SUCCESS`
3. Order status becomes `PAID`
4. Reserved stock becomes sold stock
5. Notification log is created
6. Duplicate success callback does not double update stock

US-PAY-002 Simulate Payment Failure

As a customer, I want to simulate failed payment so that the system can release reserved stock.

Acceptance Criteria

1. Payment status becomes `FAILED`
2. Order status becomes `PAYMENT_FAILED`
3. Reserved stock is released back to available stock
4. Customer can see failed status in order detail

15.6 Epic: Order Tracking

US-ORD-001 View Order History

As a customer, I want to view my order history so that I can track my purchases.

Acceptance Criteria

1. Customer sees only their own orders
2. List shows order number, date, total and status
3. List supports pagination

US-ORD-002 View Order Detail

As a customer, I want to view order detail so that I can understand item, payment and shipping status.

Acceptance Criteria

1. Detail shows item price snapshot
2. Detail shows shipping address snapshot
3. Detail shows payment status
4. Detail shows shipment status and tracking number if shipped
5. Detail shows status timeline

US-ORD-003 Cancel Pending Order

As a customer, I want to cancel unpaid order so that I do not proceed with unwanted purchase.

Acceptance Criteria

1. Customer can cancel only `PENDING_PAYMENT` order
2. Cancel releases reserved stock
3. Order status becomes `CANCELLED`
4. Cannot cancel paid, shipped, delivered or already cancelled order

15.7 Epic: Admin Product Management

US-ADM-CAT-001 Create Product

As an admin, I want to create product so that customers can buy it.

Acceptance Criteria

1. SKU must be unique
2. Price must be greater than zero
3. Category must exist
4. Initial stock creates inventory record
5. Audit log is created

US-ADM-CAT-002 Update Product

As an admin, I want to update product information so that catalog stays correct.

Acceptance Criteria

1. Admin can update name, description, price, image, visibility and status
2. SKU cannot be changed after creation unless explicitly allowed by design
3. Price update does not affect existing order item price snapshot
4. Audit log is created for sensitive updates

15.8 Epic: Admin Order Fulfillment

US-ADM-ORD-001 Manage Order Status

As an admin, I want to update order fulfillment status so that customer can track delivery.

Acceptance Criteria

1. Admin can move **PAID** to **PACKING**
2. Admin can move **PACKING** to **SHIPPED** only with tracking number
3. Admin can move **SHIPPED** to **DELIVERED**
4. Invalid transition returns **INVALID_ORDER_STATE**
5. Every transition creates status history and audit log

15.9 Epic: Review

US-REV-001 Create Product Review

As a customer, I want to review delivered products so that other customers can evaluate product quality.

Acceptance Criteria

1. Customer can review only delivered order item
2. One order item can have only one review
3. Rating must be between 1 and 5
4. Review appears in product detail if visible

16. Non-Functional Requirements

16.1 Security

ID	Requirement	Priority
SEC-001	Password must be hashed	Must
SEC-002	JWT secret must come from environment variable	Must
SEC-003	Admin APIs require admin role	Must
SEC-004	Customer resource ownership must be checked	Must
SEC-005	Input validation on all write APIs	Must
SEC-006	Do not log password, token, or sensitive data	Must
SEC-007	Use parameterized SQL or ORM safe query	Must
SEC-008	Basic rate limit on auth endpoints	Should
SEC-009	CORS config must not be wildcard in production mode	Should

16.2 Performance

ID	Requirement	Target
PERF-001	Product listing p95 latency local environment	< 300ms with 1,000 products
PERF-002	Checkout p95 latency local environment	< 500ms
PERF-003	Order detail p95 latency local environment	< 300ms
PERF-004	API pagination required for listing APIs	Required
PERF-005	Avoid N+1 query on product listing	Required

16.3 Reliability

ID	Requirement	Priority
REL-001	Checkout must be transactional	Must
REL-002	Payment success update must be transactional	Must
REL-003	Payment callback must be idempotent	Must
REL-004	Stock cannot become negative under concurrent checkout	Must
REL-005	Use optimistic or pessimistic locking for stock update	Must

16.4 Observability

ID	Requirement	Priority
OBS-001	Every API response should include requestId	Should
OBS-002	Structured logs should include requestId, userId if available, path, status, latency	Should
OBS-003	Error logs should include internal reason but not sensitive data	Must
OBS-004	Health check endpoint required	Must
OBS-005	Readiness check should verify database connection	Should

16.5 Maintainability

ID	Requirement	Priority
MAINT-001	Code must separate domain logic from HTTP handlers	Must
MAINT-002	Business rules should be covered by unit tests	Must
MAINT-003	API spec must match implementation	Must
MAINT-004	Database migrations must be versioned	Must
MAINT-005	Avoid hard-coded config	Must

17. Architecture Requirement

17.1 Recommended Architecture Style

ใช้ **modular monolith** สำหรับ MVP

เหตุผล

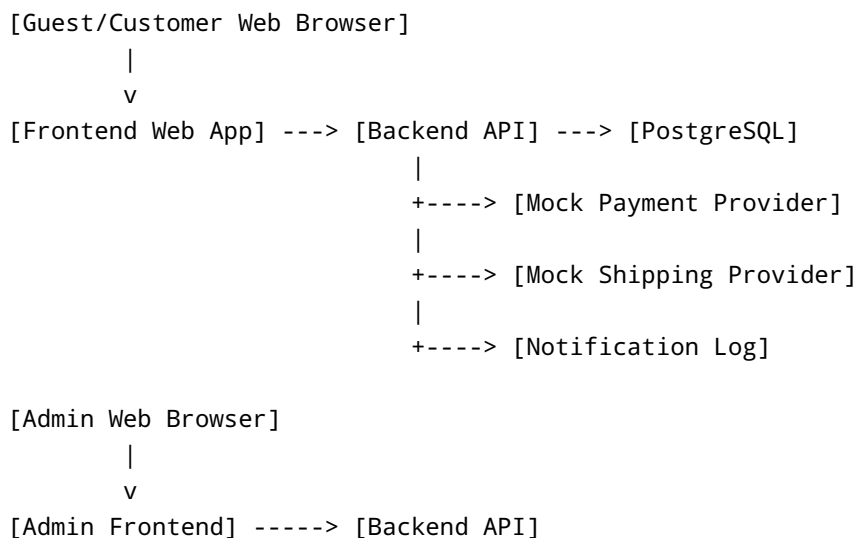
1. ลด operational complexity
2. เหมาะกับ MVP ที่ต้องส่งเร็ว
3. ยิงบังคับให้แยก domain boundary ได้
4. สามารถแยก module เป็น microservices ภายหลังได้
5. ง่ายต่อการทดสอบ agentic coding workflow เพราะ source-code repo ไม่กระเจาเกินไป

17.2 Required Architecture Document Sections

Architecture document ต้องมีอย่างน้อย

1. System context diagram
2. Container diagram
3. Component diagram ของ backend
4. Database ERD
5. Checkout sequence diagram
6. Payment success sequence diagram
7. Order fulfillment sequence diagram
8. State machine diagram สำหรับ order
9. Key architecture decisions
10. Security considerations
11. Observability design
12. Scalability considerations
13. Known limitations

17.3 Suggested Context Diagram



17.4 Checkout Sequence

```
Customer -> Frontend: Click checkout
Frontend -> Backend: POST /checkout/preview
Backend -> Cart: Load active cart
Backend -> Catalog: Validate product status
Backend -> Inventory: Check available stock
Backend -> Promotion: Validate coupon
Backend -> Backend: Calculate total
Backend -> Frontend: Preview total
Customer -> Frontend: Confirm order
Frontend -> Backend: POST /checkout/place-order
Backend -> DB Transaction: Create order + order items
Backend -> Inventory: Reserve stock
Backend -> Payment: Create payment transaction
Backend -> Cart: Mark cart checked out
Backend -> Frontend: Return order + payment intent
```

17.5 Payment Success Sequence

```
Customer -> Frontend: Simulate payment success
Frontend -> Backend: POST /payments/{id}/simulate-success
Backend -> Payment: Validate transaction
Backend -> Order: Validate order state
Backend -> Inventory: Convert reserved to sold
Backend -> Order: Change status to PAID
Backend -> Shipping: Create shipment draft
Backend -> Notification: Create payment success log
Backend -> Frontend: Return payment success result
```

18. Frontend Requirement

18.1 Customer Pages

Page	Path	Requirement
Home	/	Show featured products and categories
Product Listing	/products	Search, filter, sort, pagination
Product Detail	/products/:id	Product info, image, stock, reviews
Login	/login	Login form
Register	/register	Registration form

Page	Path	Requirement
Cart	<code>/cart</code>	View/update/remove cart items
Checkout	<code>/checkout</code>	Address, coupon, preview, place order
Payment Mock	<code>/payment/:transactionId</code>	Simulate success/failed/timeout
Order History	<code>/orders</code>	Customer order list
Order Detail	<code>/orders/:id</code>	Order detail and timeline
Profile	<code>/profile</code>	Profile and addresses

18.2 Admin Pages

Page	Path	Requirement
Admin Login	<code>/admin/login</code>	Admin login
Dashboard	<code>/admin</code>	Summary cards
Product Management	<code>/admin/products</code>	List/create/update products
Category Management	<code>/admin/categories</code>	CRUD categories
Inventory	<code>/admin/inventory</code>	Stock view and adjustment
Order Management	<code>/admin/orders</code>	Search/filter/update orders
Promotion	<code>/admin/coupons</code>	Create/update coupons
Audit Log	<code>/admin/audit-logs</code>	View audit logs

18.3 Frontend UX Rules

1. Show loading state for API calls
2. Show clear validation errors
3. Disable checkout button when cart invalid
4. Show stock warning when quantity exceeds available stock
5. Show order status badge with different visual states
6. Admin destructive actions require confirmation
7. Do not expose admin navigation to customer role

19. Testing Requirements

19.1 Unit Tests

Required coverage areas

1. Coupon discount calculation
2. Shipping fee calculation
3. Order state transition validation
4. Inventory reserve/release/confirm sold
5. Checkout total calculation
6. Payment idempotency rule
7. Product visibility rule
8. Review eligibility rule

19.2 Integration Tests

Required scenarios

1. Register → Login → Add cart → Checkout preview
2. Checkout place order reserves stock
3. Payment success confirms stock sold
4. Payment failed releases stock
5. Customer cannot access another customer order
6. Admin can update order status with valid transition
7. Admin cannot perform invalid order transition
8. Coupon usage limit works
9. Concurrent checkout cannot oversell stock

19.3 E2E Tests

Minimum E2E flows

1. Customer successful purchase flow
2. Customer payment failed flow
3. Admin create product and customer sees it
4. Admin fulfill order to delivered and customer writes review

19.4 API Contract Tests

1. Validate response shape for product listing
 2. Validate error response standard
 3. Validate auth required endpoints
 4. Validate admin-only endpoints
 5. Validate checkout and payment payloads
-

20. Seed Data Requirement

ระบบควรมี seed data สำหรับ local demo

20.1 Users

Email	Password	Role
admin@shoppilot.local	Admin@1234	ADMIN
customer1@shoppilot.local	Customer@1234	CUSTOMER
customer2@shoppilot.local	Customer@1234	CUSTOMER

20.2 Categories

1. Electronics
2. Fashion
3. Home & Living
4. Beauty
5. Sports

20.3 Products

At least 20 products

Required product patterns

1. Active product with stock
2. Active product low stock
3. Active product out of stock
4. Inactive product
5. Deleted product
6. Product with discount compare price
7. Product with multiple images
8. Product with reviews

20.4 Coupons

Code	Type	Rule
WELCOME100	FIXED	ลด 100 เมื่อซื้อครบ 500
SAVE10	PERCENTAGE	ลด 10%, สูงสุด 300
FREESHIP	FIXED	ลดไม่ได้จริง แต่ใช้ทดสอบ invalid design หรือ shipping promotion stretch
EXPIRED50	FIXED	หมดอายุแล้ว

21. Repository Deliverable Requirement

21.1 Minimum Repo Files

```
README.md
.env.example
docker-compose.yml
Makefile or taskfile.yml
apps/backend
apps/frontend
docs/user-stories/README.md
docs/architecture/system-architecture.md
docs/api/openapi.yaml
docs/database/erd.md
docs/test-plan/test-plan.md
docs/adr/0001-architecture-style.md
```

21.2 README Must Include

1. Project overview
2. Tech stack
3. Architecture summary
4. Prerequisites
5. Environment variables
6. How to run locally
7. How to run database migration
8. How to seed data
9. How to run tests
10. Demo accounts
11. API documentation path
12. Known limitations

21.3 Developer Commands

ควรมี command กลาง เช่น

```
make setup
make dev
make test
make lint
make migrate
make seed
make down
```

หรือถ้าใช้ `just`

```
just setup
just dev
just test
just lint
just migrate
just seed
just down
```

22. Documentation Requirement

22.1 User Story Document

ต้องแยกตาม epic

```
docs/user-stories/
  authentication.md
  product-catalog.md
  cart.md
  checkout.md
  payment.md
  order.md
  admin.md
  review.md
```

แต่ละ story ต้องมี

1. Story ID
2. Story statement
3. Business value
4. Acceptance criteria
5. Edge cases
6. API references
7. Test references

22.2 API Spec

ต้องใช้ OpenAPI 3.0 หรือ Markdown ที่มี

1. Endpoint
2. Method
3. Auth requirement

4. Request schema
5. Response schema
6. Error response
7. Example request/response

22.3 Architecture Document

ต้องมี

1. Overview
2. Goals and constraints
3. System context
4. Component diagram
5. Data model
6. API design principles
7. Security design
8. Transaction boundary
9. State machine
10. Scalability plan
11. Trade-offs

22.4 ADR Documents

อย่างน้อยควรมี

1. 0001-use-modular-monolith.md
2. 0002-use-postgresql.md
3. 0003-use-mock-payment-provider.md
4. 0004-use-jwt-authentication.md
5. 0005-stock-reservation-strategy.md

23. Agentic Workflow Task Breakdown

Phase 1: Product Breakdown

Output

1. User stories per epic
2. Acceptance criteria
3. Edge cases
4. Test scenario matrix

Suggested command prompt

Read `PRODUCT_REQUIREMENTS.md` and generate `docs/user-stories/*.md`. Keep story IDs stable. Add edge cases and map each story to API endpoints and tests.

Phase 2: Architecture Design

Output

1. System architecture document
2. ERD
3. Sequence diagrams
4. ADRs
5. API boundary

Suggested command prompt

Read `PRODUCT_REQUIREMENTS.md` and `docs/user-stories`. Design a modular monolith architecture for ShopPilot MVP. Generate `docs/architecture/system-architecture.md`, `docs/database/erd.md`, and ADRs.

Phase 3: API Contract

Output

1. OpenAPI spec
2. Error response standard
3. Auth rules
4. Example payloads

Suggested command prompt

Generate OpenAPI 3.0 spec for all MVP APIs from the requirement and user stories. Ensure request/response schemas match data model and error format.

Phase 4: Backend Implementation

Output

1. Backend service
2. Migration
3. Seed data
4. Unit tests
5. Integration tests

Suggested command prompt

Implement the backend for ShopPilot MVP following the generated architecture and OpenAPI spec. Prioritize domain correctness, transaction safety, stock reservation, order state machine, and payment idempotency.

Phase 5: Frontend Implementation

Output

1. Customer web UI
2. Admin web UI
3. API integration
4. Form validation
5. Basic E2E flow

Suggested command prompt

Implement the frontend for ShopPilot MVP. Build customer and admin pages, integrate with backend APIs, handle loading/error states, and support the full happy path purchase flow.

Phase 6: QA & Review

Output

1. Test plan
2. Test cases
3. Bug report
4. Security checklist
5. Architecture review

Suggested command prompt

Review the ShopPilot repo against PRODUCT_REQUIREMENTS.md. Produce a gap analysis, bug list, missing tests, security issues, and recommended fixes. Then apply safe fixes with tests.

24. Edge Cases for Agent Testing

Agents must handle these cases correctly

24.1 Checkout Edge Cases

1. Cart is empty
2. Product becomes inactive after added to cart
3. Stock decreases after item added to cart
4. Coupon valid during preview but expired before place order
5. Customer submits checkout twice
6. Customer uses address belonging to another user
7. Product price changes after item added to cart
8. Shipping fee changes based on subtotal

24.2 Payment Edge Cases

1. Payment success callback called twice
2. Payment success amount mismatch
3. Payment failed after already paid
4. Payment success after payment expired
5. Payment transaction belongs to another customer

24.3 Inventory Edge Cases

1. Two customers checkout last stock at the same time
2. Admin decreases stock below reserved quantity
3. Payment failed should release stock only once
4. Cancel order should release stock only if order is pending payment

24.4 Order Edge Cases

1. Customer tries to cancel shipped order
2. Admin tries to ship order without tracking number
3. Admin tries invalid transition from delivered to packing
4. Customer tries to view another customer order
5. Review before delivered

25. Definition of Done

MVP is done when

1. Backend and frontend can run locally with one command
2. Database migration and seed data work
3. Demo customer can complete purchase flow
4. Demo admin can create product and fulfill order
5. Payment success and failed flows update stock correctly
6. Product listing, cart, checkout, order history work
7. Admin product, inventory and order management work
8. API spec exists and mostly matches implementation
9. Architecture document exists

10. User stories exist in Markdown
 11. At least critical unit and integration tests pass
 12. Security basics are implemented
 13. README explains setup and demo flow
 14. No sensitive data is logged
 15. No known critical business rule bug remains
-

26. MVP Acceptance Demo Script

ใช้ script นี้ทดสอบ repo ที่ agent สร้าง

26.1 Setup

1. Clone repo
2. Copy `.env.example` to `.env`
3. Run `docker compose up` or `make dev`
4. Run migration
5. Run seed data
6. Open frontend
7. Login as customer
8. Login as admin in separate browser/session

26.2 Customer Happy Path

1. Customer browses product listing
2. Customer filters Electronics
3. Customer opens product detail
4. Customer adds 2 items to cart
5. Customer opens cart
6. Customer applies `WELCOME100`
7. Customer previews checkout
8. Customer places order
9. Customer simulates payment success
10. Customer sees order status `PAID`

Expected result

1. Order created
2. Payment success
3. Stock reserved then sold
4. Notification log created
5. Order visible in order history

26.3 Admin Fulfillment Path

1. Admin opens order management

2. Admin finds customer order
3. Admin changes status to `PACKING`
4. Admin changes status to `SHIPPED` with tracking number
5. Admin changes status to `DELIVERED`
6. Customer opens order detail
7. Customer writes review

Expected result

1. Order status timeline complete
2. Tracking number visible
3. Review visible on product detail
4. Audit logs created

26.4 Failure Path

1. Customer adds product to cart
2. Customer places order
3. Customer simulates payment failed
4. Customer checks order detail
5. Admin checks inventory

Expected result

1. Order status `PAYMENT_FAILED`
2. Reserved stock released
3. Product available stock restored
4. Duplicate payment failed callback does not double release stock

27. Optional Stretch Goals

ถ้า agent ทำ MVP ได้เร็ว สามารถเพิ่ม stretch goals เพื่อทดสอบความสามารถเพิ่ม

1. Redis cache for product listing
2. Outbox pattern for notification events
3. Background job for payment expiry
4. Admin dashboard chart
5. CI pipeline with test/lint
6. Playwright E2E tests
7. OpenTelemetry tracing
8. Rate limiting middleware
9. Product import CSV
10. Soft-delete recovery for product
11. Refund mock flow
12. Return request flow
13. Multi-language product fields

- 14. Feature flag for coupon module
- 15. Docker production build

28. Recommended Evaluation Rubric

Area	Weight	Evaluation
Requirement coverage	20%	Covers MVP modules and user stories
Domain correctness	20%	Stock, order state, payment idempotency correct
Code quality	15%	Clean structure, maintainable, not over-engineered
API design	10%	Consistent, documented, validated
Data model	10%	Supports business rules and history
Testing	10%	Critical flows covered
Documentation	10%	Architecture, API, README, user stories complete
Developer experience	5%	Easy setup, commands, seed data

Score Interpretation

Score	Meaning
90-100	Strong production-like MVP
75-89	Good MVP with minor gaps
60-74	Functional but missing important edge cases
40-59	CRUD-level, insufficient domain correctness
< 40	Not acceptable for workflow benchmark

29. Final Prompt for Coding Agents

ใช้ prompt นี้เป็น master prompt ได้

You are building ShopPilot MVP, a B2C single-merchant e-commerce platform.

Read PRODUCT_REQUIREMENTS.md carefully and implement a production-like MVP, not a toy CRUD app.

Required outputs:

1. Source-code repo that runs locally
2. README.md with setup/run/test/seed/demo instructions
3. User stories in docs/user-stories
4. System architecture document in docs/architecture
5. OpenAPI 3.0 spec in docs/api/openapi.yaml
6. Database schema, migrations, seed data
7. Test plan and critical tests
8. ADRs for major decisions

Architecture preference:

- Modular monolith
- Clear domain boundaries
- Domain logic separated from HTTP handlers
- PostgreSQL database
- Docker Compose for local run

Critical business rules:

- Checkout must validate cart, product status, stock, address and coupon server-side
- Stock reservation must prevent overselling
- Payment success/failed/timeout must update order and stock correctly
- Payment callback must be idempotent
- Order state transition must be enforced
- Customer must not access other customer resources
- Admin-only APIs must be protected

Do not over-engineer beyond MVP, but keep the design extensible.

Prioritize correctness of checkout, payment, inventory and order state machine.